Materiale fornito come supporto per lo studio del corso di SWAM tenuto presso UNIFI. Alcune delle figure o dei loghi presenti possono essere tratte da pubblicazioni o siti web e quindi soggette a copyright. Pertanto la distribuzione di questo materiale è lecita se effettuata a fini di insegnamento o di ricerca scientifica il cui utilizzo sia per finalità illustrative o di studio e per fini non commerciali.

A.A. 2021/2022

RESTful Architecture - Part V

Ing. Jacopo Parri, Ing. Samuele Sampietro

STLab - Software Technologies Lab

Materiale di supporto al corso di
Software Architectures and Methodologies - Laboratory

UNIVERSITÀ DEGLI STUDI FIRENZE

Alcuni JavaScript UI Framework

Per realizzare il frontend esistono molteplici tecnologie basate sul linguaggio JavaScript, oltre che HTML e CSS:

- **Angular**
- Ember.js
- jQuery
- **React.js**
- **Vue.js**
- etc.

Prima di fare un breve confronto tra questi framework **indaghiamo lo stato dell'arte**

Stato dell'Arte

Un framework JavaScript allo stato dell'arte deve essere "conforme" a certi **requisiti**:

- deve possedere un ecosistema di **librerie a supporto** di vari casi d'uso
  - e.g. routing, comunicazioni http, risoluzione automatica delle dipendenze, etc.
- dovrebbe implementare la specifica **ECMAScript**
- dovrebbe rispettare la specifica dei **Web Components**
  - https://www.webcomponents.org/specs

Web Components

L'intento principale di questa Specifica è quello di abilitare gli sviluppatori a:

- definire i propri tag
  - **Custom Elements Specification**
- i cui stili sono incapsulati e isolati
  - **Shadow DOM Specification**
- che possono essere "ristampati" molteplici volte
  - **HTML Template Specification**
- integrabili in modo consistente nelle applicazioni
  - **ECMAScript Module Specification**

in pratica

è una meta-specifica, composta da 4 specifiche

Custom Elements Specification

Getta le fondamenta per la progettazione di nuovi tipi di elementi per il Domain Object Model (**DOM**).

Per merito della specifica, gli sviluppatori possono definire gli stili e i comportamenti:

- di nuovi tag HTML
  - **autonomous custom element**
- di tag pre-esistenti
  - **customized built-in element**

```
// JavaScript code
class AutonomousButton extends HTMLElement {
    … //autonomous button behaviour
}

customElements
    .define("aut-button", AutonomousButton);

// HTML code
<aut-button>Click Me :)</aut-button>
```


Shadow DOM Specification

Il problema
All'interno del DOM, le informazioni di "stile" possono fuoriuscire da un elemento e sovrapporsi tra elementi custom e built-in del documento.

La soluzione
Le Shadow DOM API superano queste limitazioni:

- i sotto-alberi sono **isolati** e **incapsulati**; gli stili non possono essere applicati ad elementi esterni e viceversa

```
// HTML fragment
<style>
  body { color: white; }
  .test { background-color: red; }
</style>

<styled-element>
  #shadow-root
  <style>
    div { background-color: blue; }
  </style>
  <div class="test">Test</div>
</styled-element>
```


HTML Template Specification

Prevede la definizione di "elementi" template, utili a dichiarare **frammenti di HTML** che possono essere clonati e inseriti nel documento tramite **JavaScript**.

I contenuti tra i tag `<template></template>`:

- non verranno renderizzati
- non appariranno nel DOM
- non avranno effetto su altre parti di pagina
- fino ad *esplicita attivazione*

Possono essere allocati ovunque nel DOM:

- nelle sezioni `<head>`, `<body>` o `<frameset>`

```
// HTML fragment
<template id="myt">
  <img src="" alt="great image">
  <div class="comment"></div>
</template>

// JavaScript code
var t = document.querySelector('#myt');

t.content.querySelector('img').src = 'logo.png';

var clone = document.importNode(t.content, true);
document.body.appendChild(clone);
```


ECMAScript Module Specification

Definisce il modo in cui si possono includere e riusare script "esterni".

Abilita la modularizzazione di applicazioni, andando incontro alle esigenze delle Architetture Enterprise.

Gli script multipli referenziati da un frontend, vengono fusi e uniti in un unico script:

- lato client
- attraverso un pre-processing lato server

```
// HTML fragment import MODE 1
<script type="module" src="aut-things.js"></script>

// HTML fragment import MODE 2
<script type="module">
  import {autonomousButton} from 'aut-things.js';
</script>

// imported TAG usage
<aut-button>
  … // custom element example
</aut-button>
```

Osservazione

La **Specifica Web Components** permette quindi:

- di creare **"custom elements"**
  - cioè nuovi elementi HTML con una propria logica di funzionamento
- in grado di essere eseguiti ed interpretati in modo indipendente dalle tecnologie adottate
  - *framework agnostic*

NOTA i migliori framework JS e TS oltre che rispettare la Specifica favoriscono la **riusabilità**:

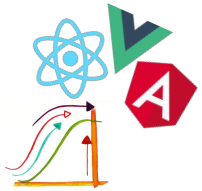
- fornendo **supporto** all'uso di "custom elements"
- permettendo **esportabilità** da "componente" del framework a "custom element" della Specifica

Perché Angular?

Le motivazioni che ci spingono a trattare Angular, anziché altri framework sono varie:

1. **STLAB** usa primariamente Angular!"
2. E' il framework **maggiormente strutturato**
   - per questo motivo, molto apprezzato per lo sviluppo di frontend in Architetture Enterprise
3. **La logica di sviluppo è rappresentativa** anche delle altre tecnologie JS
   - le interazioni frontend-backend di un'architettura REST rimangono pressoché le stesse

Angular vs React vs Vue



- Angular
  - approccio "maggiormente" enterprise e strutturato
  - con curva di apprendimento ripida
- React
  - approccio "snello" e destrutturato per UI semplici
  - con curva di apprendimento media
- Vue
  - approccio "ibrido" tra Angular e React
  - con curva di apprendimento morbida

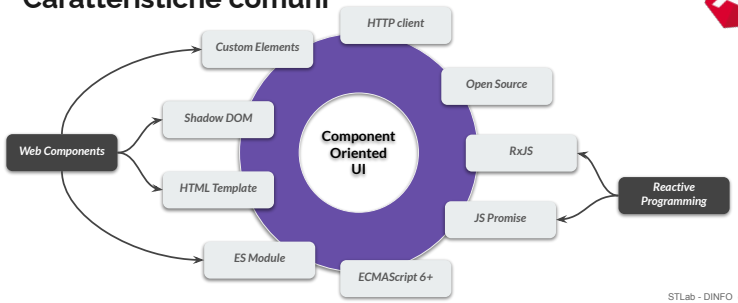
Curiosità

- Angular è il più **ostico**
- React è il più **diffuso**
- Vue è il più **apprezzato**

Angular vs React vs Vue

	Angular	React	Vue
tipo	framework	libreria	libreria/framework
popolarità	molto popolare	estremamente popolare	popolare
difficoltà	molto elevata	elevata	ridotta
sviluppato da	*Google*	*Facebook*	*community*
rilascio	*2010*	*2013*	*2014*

Caratteristiche comuni



Un “confronto in breve”

Analizziamo brevemente come i 3 principali framework trattano i concetti di:

- COMPONENTE
- TEMPLATE

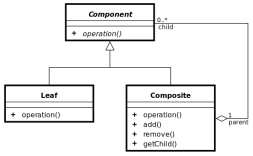
Daremo anche uno sguardo al codice

Il concetto di widget

“Un widget, in informatica, nell’ambito della programmazione, è un componente grafico di un’interfaccia utente di un programma, che ha lo scopo di facilitare all’utente l’interazione con il programma stesso”

Le interfacce utente moderne prevedono una **struttura composizionale**, in cui si definiscono elementi grafici funzionali più grandi a partire da quelli più piccoli, favorendo la riusabilità:

- questa struttura si ispira al *Design Pattern Composite*



Un componente Angular

Componente

```
@Component({
  selector: 'dog-element',
  templateUrl: './dog.component.html'})
export class DogComponent {
  dog: Dog = {
    uuid: “5579fa07-249c-4a59-a393-24d0cb986e17”,
    name: 'Fufi'
  };

  constructor() {...}

  //...
}
```

Template

```
// “./dog.component.html” file

<h2>{{dog.name}} Details</h2>

<div><span>uuid: </span>{{dog.uuid}}</div>

<form>
  <label>name:
    <input [(ngModel)]="dog.name" placeholder="Name"/>
  </label>
</form>
```

Un componente React

React

Componente

```
class Dog extends React.Component {
  constructor(props) {
    super(props);
    this.state = { uuid: '...', name: 'Fufi' };
  }
  nameHandler = (event) => {
    this.setState({name: event.target.value});
  }
  render() {...} // ----------> see template
}

ReactDOM
  .render(<Dog />, document.getElementById('root'));
```

Template

```
render() {
  return(
    <h2>{this.state.name} Details</h2>

    <div><span>uuid: </span>{this.state.uuid}</div>

    <form>
      <label>name:</label>
      <input type='text' onChange={this.nameHandler}
             placeholder="Name" />
    </form>
  );
}
```


Un componente Vue

Componente

```
<script>
  export default {
    name: 'dog',
    data: () => ({
      uuid: '5579fa07-249c-4a59-a393-24d0cb986e17',
      name: 'Fufi'
    }),
    created () {
      console.log('Authorizations page created');
    }
  }
</script>
```

Template

```
<template>
  <h2>{{ name }} Details</h2>

  <div><span>uuid: </span>{{ uuid }}</div>

  <form>
    <label>name:</label>
    <input v-model="name" placeholder="Name" />
  </form>
</template>
```


Gestione dello “stato”

Alcuni framework JavaScript hanno introdotto alcune modalità di **gestione dello stato applicativo**:

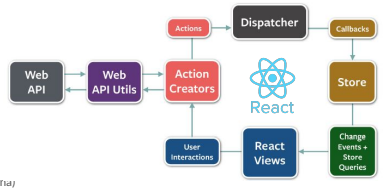
- l’idea è di poter disporre di storage lato client interni alla app
  - che non sono riferiti ai *local/session storage* del browser
- all’interno di tali “magazzini” può essere collocato lo **stato attuale di un frontend**
  - **domain state**
    - valori assunti dalle istanze di oggetti del dominio applicativo
  - **UI state**
    - elenco dei componenti “alive”
    - pagine caricate e inizializzate

FLUX *by Facebook*

Flux

FLUX è definito come un **pattern** che può essere utile a favorire lo *state management* di frontend Javascript:

- trattasi di una declinazione del pattern MVC in cui l’interazione segue un *One-way data flow*
  - **Views**
  - **Action Creators**
    - contatta le API del backend
  - **Dispatcher**
    - smista le azioni
  - **Store**
    - mantiene lo stato complessivo dell’app e della UI
    - riceve azioni che aggiornano lo stato (rappresentano l’unico modo in cui si aggiorna)
    - notifica i cambiamenti



Redux

Redux si autodefinisce:

"contenitore di stato predicibile"

- in realtà, può essere considerata una **implementazione evolutiva del pattern FLUX**, indipendente dalla tecnologia adottata per lo strato delle viste
- Redux offre un **approccio Enterprise**, maggiormente strutturato per *framework lightweight* (e.g. React e Vue)

```
import { createStore } from 'redux'
function counter(state = 0, action) {
  switch (action.type) {
    case 'INCREMENT':
      return state + 1
    case 'DECREMENT':
      return state - 1
    default:
      return state
  }
}
```

```
let store = createStore(counter)

store.subscribe(() => console.log(store.getState()))

store.dispatch({ type: 'INCREMENT' })// 1
store.dispatch({ type: 'INCREMENT' })// 2
store.dispatch({ type: 'DECREMENT' })// 1
```


Tirare le somme...

Il parere su DZone

https://dzone.com/articles/angular-vs-react-vs-vue-which-framework-is-best-to

Angular

Angular is best suitable for large-scale and advanced projects. These may include but not be limited to:

- Developing Progressive Web App (PWA).
- Redesigning of a website application.
- Building dynamic content-based web designs.
- Creating large enterprise application that needs complex infrastructure.

Il parere su DZone

https://dzone.com/articles/angular-vs-react-vs-vue-which-framework-is-best-to

React

MERN = MongoDB, Express, React, Node

React comes from the family of the MERN stack — a technology stack known for building sophisticated business applications. React becomes a powerful tool when used along with Redux or MobX, or any flux patterned library. React is best suited for the following projects:

- For applications that involve many components with navigation items, collapsed/expanded accordion sections, active/inactive states, dynamic inputs, active/inactive buttons, user login, user access permissions, etc.
- For projects that have scope for scale and growth because React components have declarative nature that makes way for easy handling around such complex structures.
- When UI is the center of the web app.

Il parere su DZone

https://dzone.com/articles/angular-vs-react-vs-vue-which-framework-is-best-to

Vue

Vue is best suited for solving short-term problems because Vue has an affordable and quick learning curve. It can integrate with existing code blocks easily. Vue may be required when:

- You need development projects of web apps with animations or interactive elements.
- Prototyping without the need for advanced skills.
- Applications requiring seamless integration with multiple other apps.
- Faster launch of MVP.

MVP = Minimum Viable Product

Prossimamente...

Angular Framework

Approfondimenti

- https://www.ibm.com/developerworks/library/ws-restful/index.html
- https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods
- https://en.wikipedia.org/wiki/List_of_HTTP_status_codes
- https://www.w3.org/TR/soap/
- https://martinfowler.com/articles/richardsonMaturityModel.html
- https://www.crummy.com/writing/speaking/2008-QCon/act3.html
- Fowler, Martin. UML distilled: a brief guide to the standard object modeling language. Addison-Wesley Professional, 2004
- http://www.oracle.com/technetwork/java/dataaccessobject-138824.html
- https://martinfowler.com/eaaCatalog/dataTransferObject.html
- http://wildfly.org/
- https://jcp.org/en/jsr/detail?id=311
- http://resteasy.jboss.org/
- https://oauth.net/
- https://oauth.net/2/
- https://tools.ietf.org/html/rfc7519
- https://www.jcp.org/en/jsr/detail?id=330
- https://martinfowler.com/articles/injection.html
- https://docs.oracle.com/javaee/6/tutorial/doc/gkeed.html
- https://www.getpostman.com/
- http://junit.org/junit5/
- http://rest-assured.io/
- http://www.oracle.com/technetwork/java/javaee/javaserverfaces-139869.html
- http://reactivex.io/rxjs/
- http://es6-features.org/
- https://www.typescriptlang.org/
- https://www.typescriptlang.org/docs/handbook/advanced-types.html
- https://developer.chrome.com/apps/manifest/kiosk_enabled
- https://developers.google.com/web/progressive-web-apps/
- https://facebook.github.io/flux/
- https://dev.to/duomly/the-best-front-end-framework-to-learn-in-2019-dn7